



UNIVERSIDAD PERUANA LOS ANDES

Facultad de Ingeniería — Escuela Profesional de Ingeniería de Sistemas y Computación



ACTIVIDAD — BASE DE DATOS II

Semana 11

Estudiante: Eduardo Fredy Ramón Puente

Código: T01311A

Curso: Base de Datos II

Docente: Ing. Raúl Fernández Bejarano



TEMA 1. Autenticación SQL y Windows: diferencias y prácticas seguras

Proyecto 1: Implementación de acceso mixto para personal académico y administrativo

1. Enunciado del ejercicio

La Universidad requiere que los trabajadores internos accedan mediante autenticación Windows, mientras que los consultores externos utilicen autenticación SQL Server. Diseñe e implemente una estrategia de autenticación para la base de datos GradosTítulos que permita administrar ambos tipos de acceso.

2. Script de la solución en T-SQL

```
USE master;
GO

-- Login para personal interno (autenticacion de Windows / Active Directory)
CREATE LOGIN [UPLA\PersonalAcademico] FROM WINDOWS;
GO

-- Login para consultores externos (autenticacion SQL Server)
CREATE LOGIN ConsultorExterno01
    WITH PASSWORD = 'C0ns_2026#Segura!',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON;
GO

USE GradosTítulos;
GO

CREATE USER [UPLA\PersonalAcademico] FOR LOGIN [UPLA\PersonalAcademico];
CREATE USER ConsultorExterno01 FOR LOGIN ConsultorExterno01;
GO

-- Asignacion de permisos base por tipo de usuario
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Tramite TO [UPLA\PersonalAcademico];
GRANT SELECT ON SCHEMA::Catalogo TO ConsultorExterno01;
GO
```

3. Justificación técnica de la solución aplicada

El servidor se configura en modo de autenticación mixta (SQL Server and Windows Authentication mode) porque conviven dos poblaciones de usuarios: personal interno con cuenta de dominio y consultores externos sin acceso a Active Directory. La autenticación de Windows delega la validación de credenciales al controlador de dominio (Kerberos/NTLM), mientras que la autenticación SQL crea un login propio dentro del motor de base de datos para quienes no pertenecen al dominio institucional.

4. Buenas prácticas utilizadas en el proyecto

- Priorizar autenticación de Windows para el personal interno, ya que aprovecha las políticas de contraseña y bloqueo de cuentas ya administradas por el dominio.
- Aplicar CHECK_POLICY = ON y CHECK_EXPIRATION = ON a todo login SQL para forzar complejidad y caducidad de contraseña.



- Asignar permisos mínimos necesarios (principio de menor privilegio) al crear el usuario, en lugar de otorgar roles amplios por defecto.
- No utilizar la cuenta 'sa' para operaciones cotidianas ni para los consultores externos.

Proyecto 2: Gestión de accesos para miembros del Jurado de Tesis

1. Enunciado del ejercicio

Los miembros del jurado necesitan acceder únicamente a información de tesis y sustentaciones. Configure accesos diferenciados utilizando autenticación Windows y SQL.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

-- Login Windows para jurado interno (docentes UPLA)
CREATE LOGIN [UPLA\JuradoInterno01] FROM WINDOWS;
CREATE USER [UPLA\JuradoInterno01] FOR LOGIN [UPLA\JuradoInterno01];
GO

-- Login SQL para jurado externo (invitado sin cuenta de dominio)
CREATE LOGIN JuradoExterno01
    WITH PASSWORD = 'Jur@doExt_2026!',
    CHECK_POLICY = ON;
CREATE USER JuradoExterno01 FOR LOGIN JuradoExterno01;
GO

-- Rol personalizado restringido solo al esquema de evaluacion
CREATE ROLE rol_Jurado;
GRANT SELECT ON SCHEMA::Evaluacion TO rol_Jurado;
DENY INSERT, UPDATE, DELETE ON SCHEMA::Evaluacion TO rol_Jurado;
GO

ALTER ROLE rol_Jurado ADD MEMBER [UPLA\JuradoInterno01];
ALTER ROLE rol_Jurado ADD MEMBER JuradoExterno01;
GO
```

3. Justificación técnica de la solución aplicada

Se crea un rol de base de datos (rol_Jurado) que centraliza el permiso de solo lectura sobre el esquema Evaluacion, independientemente del tipo de autenticación del miembro del jurado. Esto separa la gestión de identidad (quién es el usuario) de la gestión de autorización (qué puede hacer), evitando otorgar permisos individualmente a cada login.

4. Buenas prácticas utilizadas en el proyecto

- Usar roles de base de datos en lugar de asignar permisos directamente a cada usuario, facilitando el mantenimiento y la auditoría.
- Aplicar DENY explícito sobre operaciones de escritura para dejar sin ambigüedad la restricción, incluso si el usuario perteneciera a otro rol con permisos de escritura.
- Diferenciar el mecanismo de autenticación (Windows vs SQL) sin diferenciar el nivel de autorización, ya que el criterio de acceso debe basarse en el rol funcional, no en el origen de la cuenta.

Proyecto 3: Gestión de accesos para miembros del Jurado de Tesis (políticas de cuentas SQL)

1. Enunciado del ejercicio



La Oficina de Tecnologías de Información requiere aplicar políticas estrictas a las cuentas SQL Server que administran los trámites de grados y títulos.

2. Script de la solución en T-SQL

```
-- Aplicar politicas estrictas a cuentas SQL existentes
ALTER LOGIN ConsultorExterno01 WITH CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
ALTER LOGIN JuradoExterno01 WITH CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
GO

-- Deshabilitar la cuenta 'sa' como practica de endurecimiento del servidor
ALTER LOGIN sa DISABLE;
GO

-- Trigger de servidor para restringir el inicio de sesion de cuentas SQL
-- fuera del horario administrativo (8:00 - 20:00)
CREATE TRIGGER trg_RestringirHorarioAcceso
ON ALL SERVER FOR LOGON
AS
BEGIN
    IF ORIGINAL_LOGIN() IN ('ConsultorExterno01', 'JuradoExterno01')
        AND (DATEPART(HOUR, GETDATE()) < 8 OR DATEPART(HOUR, GETDATE()) > 20)
    BEGIN
        ROLLBACK;
    END
END;
GO
```

3. Justificación técnica de la solución aplicada

Los login triggers permiten aplicar controles adicionales que van más allá de la autenticación básica, como restringir el horario de conexión de cuentas con acceso a información sensible de trámites. Deshabilitar 'sa' reduce la superficie de ataque, ya que es el objetivo más común en intentos de fuerza bruta contra SQL Server.

4. Buenas prácticas utilizadas en el proyecto

- Deshabilitar o renombrar la cuenta 'sa' y nunca usarla para tareas operativas.
- Forzar CHECK_POLICY y CHECK_EXPIRATION en todas las cuentas SQL sin excepción.
- Utilizar login triggers para reglas de negocio de acceso (horario, origen de IP) cuando el control nativo del motor no sea suficiente.
- Documentar y revisar periódicamente las cuentas SQL activas para detectar accesos que ya no deberían existir.



TEMA 2. Cuentas de servicio y configuración del servidor

Proyecto 1: Configuración segura de servicios SQL Server

1. Enunciado del ejercicio

La base de datos GradosTítulos será utilizada en producción. Configure cuentas de servicio específicas para SQL Server y SQL Server Agent.

2. Script de la solución en T-SQL

```
-- La asignacion de la cuenta de servicio (dominio\svc_sqlserver) se realiza desde
-- SQL Server Configuration Manager, no mediante T-SQL puro. Desde T-SQL se puede
-- verificar y endurecer la configuracion asociada:

-- Verificar la cuenta de servicio actual del motor
EXEC xp_instance_regread
    N'HKEY_LOCAL_MACHINE',
    N'SYSTEM\CurrentControlSet\Services\MSSQLSERVER',
    N'ObjectName';
GO

-- Deshabilitar xp_cmdshell para reducir el privilegio efectivo de la cuenta de servicio
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 0;
RECONFIGURE;
GO

-- Revisar quienes tienen el rol sysadmin (deben ser minimos)
SELECT sp.name AS Login, sp.type_desc
FROM sys.server_role_members rm
JOIN sys.server_principals sp ON rm.member_principal_id = sp.principal_id
JOIN sys.server_principals r ON rm.role_principal_id = r.principal_id
WHERE r.name = 'sysadmin';
GO
```

3. Justificación técnica de la solución aplicada

SQL Server y SQL Server Agent deben ejecutarse con cuentas de servicio dedicadas (no con la cuenta del sistema local ni con una cuenta de administrador de dominio), de modo que si el proceso es comprometido, el atacante obtenga únicamente los privilegios mínimos asignados a esa cuenta y no acceso administrativo al servidor.

4. Buenas prácticas utilizadas en el proyecto

- Usar cuentas de servicio administradas (gMSA) o cuentas de dominio dedicadas con contraseña gestionada automáticamente.
- Nunca ejecutar los servicios de SQL Server con una cuenta de administrador del dominio.
- Deshabilitar componentes de alto riesgo como xp_cmdshell si no son requeridos por el negocio.
- Revisar periódicamente la membresía del rol sysadmin y del grupo de administradores locales.

Proyecto 2: Monitoreo de servicios críticos del servidor

1. Enunciado del ejercicio



Implemente consultas administrativas que permitan verificar el estado de las instancias SQL Server relacionadas con GradosTítulos.

2. Script de la solución en T-SQL

```
-- Estado de los servicios de SQL Server visibles desde el motor
SELECT servicename, status_desc, process_id, last_startup_time
FROM sys.dm_server_services;
GO

-- Estado general de la base de datos GradosTítulos
SELECT name, state_desc, recovery_model_desc
FROM sys.databases
WHERE name = 'GradosTítulos';
GO

-- Alerta de SQL Server Agent ante errores de severidad alta (19-25)
USE msdb;
GO
EXEC dbo.sp_add_alert
    @name = N'Alerta_Servicio_GradosTítulos',
    @message_id = 0,
    @severity = 19,
    @notification_message = N'Se detecto un error critico de severidad alta relacionado con GradosTítulos.';
GO
```

3. Justificación técnica de la solución aplicada

La vista de administración dinámica sys.dm_server_services permite monitorear el estado del propio motor y del agente sin depender de herramientas externas, mientras que las alertas de SQL Server Agent automatizan la detección de errores graves (severidad 19 en adelante) que podrían indicar caída de servicio o corrupción.

4. Buenas prácticas utilizadas en el proyecto

- Configurar Database Mail para que las alertas lleguen al equipo de infraestructura en tiempo real.
- Monitorear tanto el estado del servicio como el estado de la base de datos (ONLINE, RESTORING, SUSPECT), ya que un servicio activo no garantiza una base de datos disponible.
- Registrar un historial de disponibilidad para poder reportar el cumplimiento del SLA del sistema.

Proyecto 3: Configuración de protocolos de red seguros

1. Enunciado del ejercicio

Configure la instancia SQL Server para permitir únicamente conexiones seguras a la base de datos GradosTítulos.

2. Script de la solución en T-SQL



```
-- El forzado de cifrado (Force Encryption) se activa en SQL Server Configuration Manager,  
-- pero se puede auditar el estado de las conexiones activas desde T-SQL:  
  
SELECT session_id, encrypt_option, auth_scheme, client_net_address, protocol_type  
FROM sys.dm_exec_connections  
WHERE encrypt_option = 'FALSE';  
GO  
  
-- Verificar el certificado disponible para cifrar las conexiones cliente-servidor  
SELECT thumbprint, subject, expiry_date  
FROM sys.certificates  
WHERE name LIKE '%GradosTítulos%';  
GO
```

3. Justificación técnica de la solución aplicada

Restringir la instancia a protocolos y conexiones cifradas (TLS) evita que las credenciales y los datos de trámites académicos viajen en texto plano por la red. La consulta sobre sys.dm_exec_connections permite auditar si existen sesiones activas que no están usando cifrado, lo cual indicaría una configuración incompleta.

4. Buenas prácticas utilizadas en el proyecto

- Deshabilitar protocolos heredados e inseguros (por ejemplo, versiones antiguas de TLS) a nivel de sistema operativo.
- Forzar el cifrado de todas las conexiones entrantes (Force Encryption = Yes) para la instancia.
- Usar un certificado emitido por una entidad certificadora confiable en lugar de un certificado autofirmado en ambientes de producción.



TEMA 3. Creación de roles fijos y personalizados

Proyecto 1: Roles para el proceso de grados y títulos

1. Enunciado del ejercicio

Diseñe roles personalizados para gestionar el acceso a los esquemas: *Catalogo*, *Academico*, *Tramite*, *Evaluacion* y *Documento*.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

CREATE ROLE rol_Catalogo;
CREATE ROLE rol_Academico;
CREATE ROLE rol_Tramite;
CREATE ROLE rol_Evaluacion;
CREATE ROLE rol_Documento;
GO

GRANT SELECT, INSERT, UPDATE ON SCHEMA::Catalogo TO rol_Catalogo;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Academico TO rol_Academico;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Tramite TO rol_Tramite;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Evaluacion TO rol_Evaluacion;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Documento TO rol_Documento;
GO
```

3. Justificación técnica de la solución aplicada

Al crear un rol por cada esquema funcional del sistema, el acceso queda alineado con los módulos de negocio del proceso de grados y títulos. Un usuario que trabaja solo con trámites recibe únicamente rol_Tramite, sin exponer datos de evaluación o documentos oficiales que no le corresponden.

4. Buenas prácticas utilizadas en el proyecto

- Diseñar los roles a partir de los esquemas de la base de datos y no de personas específicas, para que el modelo escale al incorporar nuevos usuarios.
- Evitar otorgar permisos a nivel de base de datos completa cuando el acceso puede acotarse a un esquema.
- Documentar la matriz de roles y esquemas para facilitar auditorías futuras.

Proyecto 2: Implementación de roles para evaluadores de tesis

1. Enunciado del ejercicio

Los evaluadores deben consultar tesis y registrar observaciones sin modificar información académica.

2. Script de la solución en T-SQL



```
USE GradosTítulos;
GO

CREATE ROLE rol_EvaluadorTesis;
GO

-- Lectura completa del esquema de evaluacion
GRANT SELECT ON SCHEMA::Evaluacion TO rol_EvaluadorTesis;

-- Permiso puntual para registrar observaciones (tabla especifica)
GRANT INSERT ON Evaluacion.Observaciones TO rol_EvaluadorTesis;

-- Se impide explícitamente modificar informacion academica
DENY UPDATE, DELETE ON SCHEMA::Academico TO rol_EvaluadorTesis;
GO
```

3. Justificación técnica de la solución aplicada

El permiso se otorga de forma diferenciada: lectura amplia sobre Evaluacion, pero escritura acotada únicamente a la tabla de observaciones. El DENY sobre el esquema Academico blindo la información curricular del estudiante, que no forma parte de las responsabilidades del evaluador.

4. Buenas prácticas utilizadas en el proyecto

- Conceder INSERT solo sobre la tabla puntual que el rol necesita modificar, en vez de todo el esquema.
- Usar DENY para dejar constancia explícita de una restricción de negocio, incluso si ningún otro permiso la habilitaría por defecto.
- Revisar que las tablas con datos académicos sensibles no queden accesibles por herencia de permisos de otros roles.

Proyecto 3: Administración de roles jerárquicos

1. Enunciado del ejercicio

Configure roles que representen los niveles: Operador, Supervisor y Administrador para el sistema de grados y títulos.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

CREATE ROLE rol_Operador;
CREATE ROLE rol_Supervisor;
CREATE ROLE rol_Administrador;
GO

-- Nivel Operador: solo consulta de tramites
GRANT SELECT ON SCHEMA::Tramite TO rol_Operador;

-- Nivel Supervisor: hereda lo del Operador (rol anidado) y agrega escritura
ALTER ROLE rol_Supervisor ADD MEMBER rol_Operador;
GRANT INSERT, UPDATE ON SCHEMA::Tramite TO rol_Supervisor;

-- Nivel Administrador: hereda del Supervisor y controla toda la base de datos
ALTER ROLE rol_Administrador ADD MEMBER rol_Supervisor;
GRANT CONTROL ON DATABASE::GradosTítulos TO rol_Administrador;
GO
```



3. Justificación técnica de la solución aplicada

SQL Server permite anidar roles de base de datos (un rol puede ser miembro de otro rol), lo cual modela de forma natural una jerarquía de permisos: Administrador incluye todo lo de Supervisor, y Supervisor incluye todo lo de Operador, evitando duplicar la asignación de permisos en cada nivel.

4. Buenas prácticas utilizadas en el proyecto

- Aprovechar la herencia de roles (`ALTER ROLE ... ADD MEMBER`) para modelar jerarquías en lugar de repetir los mismos `GRANT` en cada rol.
- Otorgar `CONTROL` total de la base de datos solo al nivel más alto (Administrador) y de forma restringida a muy pocos usuarios.
- Probar la jerarquía con un usuario de prueba antes de asignarla a cuentas reales, verificando que cada nivel reciba exactamente los permisos esperados.



TEMA 4. Control de acceso mediante GRANT, DENY y REVOKE

Proyecto 1: Roles para el proceso de grados y títulos

1. Enunciado del ejercicio

Los operadores pueden consultar datos de estudiantes, pero no modificarlos.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

GRANT SELECT ON Academico.Estudiante TO rol_Operador;
DENY INSERT, UPDATE, DELETE ON Academico.Estudiante TO rol_Operador;
GO
```

3. Justificación técnica de la solución aplicada

Se combina GRANT y DENY sobre el mismo objeto para dejar sin ambigüedad la regla de negocio: el operador puede leer, pero jamás modificar los datos del estudiante. En SQL Server, DENY tiene prioridad sobre cualquier GRANT que el usuario pudiera recibir por pertenecer a otro rol, lo que hace de este mecanismo un control robusto.

4. Buenas prácticas utilizadas en el proyecto

- Usar DENY (y no solo omitir el GRANT) cuando la restricción es una regla de negocio crítica, para blindarla frente a futuras asignaciones de roles adicionales.
- Aplicar el permiso al nivel de objeto (tabla) cuando la regla afecta a una tabla puntual, en lugar de a todo el esquema.
- Probar el acceso con la cuenta real del operador para confirmar que la restricción funciona como se espera.

Proyecto 2: Restricción de acceso a resoluciones y diplomas

1. Enunciado del ejercicio

Solo la Oficina de Grados y Títulos puede modificar documentos oficiales.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

CREATE ROLE rol_OficinaGradosTítulos;
GO

GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Documento TO rol_OficinaGradosTítulos;

-- El resto de roles operativos queda explícitamente sin permiso de escritura
DENY INSERT, UPDATE, DELETE ON SCHEMA::Documento TO rol_Operador;
DENY INSERT, UPDATE, DELETE ON SCHEMA::Documento TO rol_Supervisor;
GO
```

3. Justificación técnica de la solución aplicada



El esquema Documento concentra resoluciones y diplomas, información con valor legal. Centralizar el control total en un único rol (rol_OficinaGradosTítulos) y denegar explícitamente la escritura a los demás roles evita modificaciones no autorizadas de documentos oficiales, incluso ante errores de asignación de roles.

4. Buenas prácticas utilizadas en el proyecto

- Aislar en un esquema separado la información con valor legal o institucional (documentos oficiales) para poder aplicarle controles de acceso más estrictos.
- Aplicar el principio de segregación de funciones: quien genera el trámite no es necesariamente quien puede emitir o modificar el documento final.
- Registrar en auditoría cualquier cambio realizado sobre el esquema Documento (ver Tema 6).

Proyecto 3: Revocación de permisos temporales

1. Enunciado del ejercicio

Un auditor externo requiere acceso temporal durante una semana.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

CREATE LOGIN AuditorExterno01
    WITH PASSWORD = 'Aud_Temp2026!', CHECK_POLICY = ON;
CREATE USER AuditorExterno01 FOR LOGIN AuditorExterno01;
GO

GRANT SELECT ON SCHEMA::Tramite TO AuditorExterno01;
GO

-- Job de SQL Server Agent que revoca el acceso automaticamente pasada la semana
USE msdb;
GO
EXEC dbo.sp_add_job @job_name = N'Revocar_Acceso_AuditorExterno01';

EXEC dbo.sp_add_jobstep
    @job_name = N'Revocar_Acceso_AuditorExterno01',
    @step_name = N'RevocarPermiso',
    @subsystem = N'TSQL',
    @command = N'USE GradosTítulos; REVOKE SELECT ON SCHEMA::Tramite FROM AuditorExterno01;';

EXEC dbo.sp_add_jobschedule
    @job_name = N'Revocar_Acceso_AuditorExterno01',
    @name = N'UnaSemanaDespues',
    @freq_type = 1, -- ejecucion unica
    @active_start_date = 20260710; -- fecha = hoy + 7 dias

EXEC dbo.sp_add_jobserver
    @job_name = N'Revocar_Acceso_AuditorExterno01';
GO
```

3. Justificación técnica de la solución aplicada

En lugar de confiar en que un administrador recuerde revocar manualmente el acceso, se programa un Job de SQL Server Agent con ejecución única que aplica el REVOKE automáticamente al vencer el plazo autorizado, reduciendo el riesgo de accesos olvidados y vigentes más allá de lo necesario.

4. Buenas prácticas utilizadas en el proyecto



- Definir de antemano la fecha de expiración del acceso y automatizar su revocación en vez de depender de un recordatorio manual.
- Otorgar al auditor solo el esquema estrictamente necesario (Tramite) y no acceso general a la base de datos.
- Deshabilitar o eliminar el login del auditor una vez concluida la revisión, además de revocar los permisos sobre los objetos.



TEMA 5. Cifrado y protección de datos (TDE y Always Encrypted)

Proyecto 1: Implementación de TDE para GradosTítulos

1. Enunciado del ejercicio

Proteja los archivos físicos de la base de datos GradosTítulos mediante Transparent Data Encryption.

2. Script de la solución en T-SQL

```
USE master;
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'M@sterKey_GradosTítulos2026!';
GO
CREATE CERTIFICATE CertTDE_GradosTítulos
    WITH SUBJECT = 'Certificado para TDE de GradosTítulos';
GO

USE GradosTítulos;
GO
CREATE DATABASE ENCRYPTION KEY
    WITH ALGORITHM = AES_256
    ENCRYPTION BY SERVER CERTIFICATE CertTDE_GradosTítulos;
GO

ALTER DATABASE GradosTítulos SET ENCRYPTION ON;
GO

-- Verificar el estado del cifrado
SELECT DB_NAME(database_id) AS BaseDatos, encryption_state, encryption_state_desc
FROM sys.dm_database_encryption_keys;
GO

-- IMPORTANTE: respaldar el certificado y su llave privada de inmediato
BACKUP CERTIFICATE CertTDE_GradosTítulos
TO FILE = 'C:\Backups\CertTDE_GradosTítulos.cer'
WITH PRIVATE KEY (
    FILE = 'C:\Backups\CertTDE_GradosTítulos.pvk',
    ENCRYPTION BY PASSWORD = 'Backup_Cert_2026!'
);
GO
```

3. Justificación técnica de la solución aplicada

TDE cifra en reposo los archivos físicos .mdf y .ldf y los respaldos, protegiendo la información ante el robo del archivo o del backup, sin requerir cambios en las aplicaciones que consultan la base de datos, ya que el cifrado y descifrado es transparente para las consultas.

4. Buenas prácticas utilizadas en el proyecto

- Respalidar inmediatamente el certificado y la llave privada usados para TDE: sin ellos, un backup cifrado no puede restaurarse jamás, ni siquiera por el propio administrador.
- Almacenar el respaldo del certificado en una ubicación distinta al servidor de base de datos.
- Usar AES_256 en lugar de algoritmos más antiguos y débiles.
- Recordar que TDE protege datos en reposo, no datos en memoria ni en tránsito; debe combinarse con cifrado de red (Tema 2) y, si aplica, Always Encrypted (Proyecto 2).

Proyecto 2: Protección de datos personales del tesista (Always Encrypted)

1. Enunciado del ejercicio

Implemente Always Encrypted para proteger DNI, correo electrónico y teléfono de los estudiantes.

2. Script de la solución en T-SQL

```
-- La generacion de la Column Master Key (CMK) normalmente se realiza con el asistente
-- de SSMS o PowerShell, ya que requiere un almacen de claves externo (Windows Certificate
-- Store o Azure Key Vault). El siguiente script aplica el cifrado una vez generadas las claves:

USE GradosTítulos;
GO

CREATE COLUMN MASTER KEY CMK_Estudiante
WITH (
    KEY_STORE_PROVIDER_NAME = 'MSSQL_CERTIFICATE_STORE',
    KEY_PATH = 'CurrentUser/My/<ThumbprintDelCertificado>'
);
GO

CREATE COLUMN ENCRYPTION KEY CEK_Estudiante
WITH VALUES (
    COLUMN_MASTER_KEY = CMK_Estudiante,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x01A2B3... -- valor generado por el asistente de cifrado
);
GO

-- DNI: cifrado deterministico (permite busquedas exactas / indices)
ALTER TABLE Academico.Estudiante
ALTER COLUMN DNI CHAR(8) COLLATE Latin1_General_BIN2
ENCRYPTED WITH (
    COLUMN_ENCRYPTION_KEY = CEK_Estudiante,
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO

-- Correo y telefono: cifrado aleatorio (mayor seguridad, sin busqueda exacta)
ALTER TABLE Academico.Estudiante
ALTER COLUMN CorreoElectronico NVARCHAR(100) COLLATE Latin1_General_BIN2
ENCRYPTED WITH (
    COLUMN_ENCRYPTION_KEY = CEK_Estudiante,
    ENCRYPTION_TYPE = RANDOMIZED,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
);
GO
```

3. Justificación técnica de la solución aplicada

Always Encrypted cifra los datos en el cliente antes de que lleguen al servidor, de modo que ni el administrador de la base de datos puede ver el valor en texto plano de columnas como el DNI. Se usa DETERMINISTIC en el DNI porque debe poder buscarse por valor exacto, y RANDOMIZED en correo y teléfono, que no requieren búsquedas exactas y así obtienen mayor nivel de protección.

4. Buenas prácticas utilizadas en el proyecto



- Elegir DETERMINISTIC solo cuando la columna realmente necesita comparaciones de igualdad; en caso contrario preferir RANDOMIZED por ser criptográficamente más fuerte.
- Restringir el acceso a la Column Master Key (certificado o Key Vault) a las aplicaciones y personas estrictamente autorizadas a leer datos personales en claro.
- Combinar Always Encrypted con TDE: TDE protege el archivo físico, Always Encrypted protege la columna incluso de quien administra el motor de base de datos.

Proyecto 3: Cifrado de información de jurados y asesores

1. Enunciado del ejercicio

Proteja la información personal de asesores y jurados de tesis.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'M@sterKey_JuradosAsesores2026!';
GO
CREATE CERTIFICATE CertJuradosAsesores
    WITH SUBJECT = 'Cifrado de datos personales de jurados y asesores';
GO
CREATE SYMMETRIC KEY SK_JuradosAsesores
    WITH ALGORITHM = AES_256
    ENCRYPTION BY CERTIFICATE CertJuradosAsesores;
GO

-- Cifrado a nivel de celda para el telefono del jurado
OPEN SYMMETRIC KEY SK_JuradosAsesores
    DECRYPTION BY CERTIFICATE CertJuradosAsesores;

UPDATE Evaluacion.Jurado
SET TelefonoCifrado = EncryptByKey(Key_GUID('SK_JuradosAsesores'), Telefono);

CLOSE SYMMETRIC KEY SK_JuradosAsesores;
GO

-- Ejemplo de lectura descifrada (solo para usuarios autorizados)
OPEN SYMMETRIC KEY SK_JuradosAsesores
    DECRYPTION BY CERTIFICATE CertJuradosAsesores;

SELECT NombreJurado,
    CONVERT(VARCHAR(20), DecryptByKey(TelefonoCifrado)) AS Telefono
FROM Evaluacion.Jurado;

CLOSE SYMMETRIC KEY SK_JuradosAsesores;
GO
```

3. Justificación técnica de la solución aplicada

Cuando no se dispone de un cliente compatible con Always Encrypted para todas las aplicaciones que consumen los datos de jurados y asesores, el cifrado a nivel de celda (Cell-Level Encryption) con clave simétrica protegida por certificado ofrece una alternativa dentro del motor de base de datos: el dato solo puede descifrarse abriendo explícitamente la llave simétrica correspondiente.

4. Buenas prácticas utilizadas en el proyecto

- Limitar quién tiene permiso para abrir la llave simétrica (OPEN SYMMETRIC KEY), ya que quien puede abrirla puede leer el dato en claro.



- Cerrar siempre la llave simétrica (CLOSE SYMMETRIC KEY) inmediatamente después de usarla.
- Preferir Always Encrypted (Proyecto 2) cuando el cliente lo soporte, ya que evita exponer el dato en claro incluso dentro del motor; usar cifrado por celda como alternativa cuando esto no sea posible.



TEMA 6. Auditoría y monitoreo de eventos con SQL Server Audit

Proyecto 1: Auditoría de accesos a la base de datos

1. Enunciado del ejercicio

Registrar todos los accesos realizados a la base de datos GradosTítulos.

2. Script de la solución en T-SQL

```
USE master;
GO
CREATE SERVER AUDIT Audit_AccesosGradosTítulos
TO FILE (FILEPATH = 'C:\SQLAudit\GradosTítulos\')
WITH (ON_FAILURE = CONTINUE);
GO
ALTER SERVER AUDIT Audit_AccesosGradosTítulos WITH (STATE = ON);
GO

USE GradosTítulos;
GO
CREATE DATABASE AUDIT SPECIFICATION AuditSpec_Accesos
FOR SERVER AUDIT Audit_AccesosGradosTítulos
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (FAILED_LOGIN_GROUP),
ADD (DATABASE_LOGOUT_GROUP)
WITH (STATE = ON);
GO
```

3. Justificación técnica de la solución aplicada

SQL Server Audit permite registrar de forma nativa, a nivel de servidor y de base de datos, los eventos de inicio y cierre de sesión, tanto exitosos como fallidos. Esto genera evidencia objetiva de quién accedió a GradosTítulos y cuándo, requisito básico para cualquier control de seguridad institucional.

4. Buenas prácticas utilizadas en el proyecto

- Almacenar los archivos de auditoría en una ruta distinta al servidor de base de datos para evitar que sean alterados junto con el propio ataque.
- Registrar tanto los inicios de sesión exitosos como los fallidos, ya que estos últimos son indicadores tempranos de intentos de intrusión.
- Definir una política de retención y revisión periódica de los archivos de auditoría.

Proyecto 2: Auditoría de modificaciones en trámites académicos

1. Enunciado del ejercicio

Registrar todas las operaciones INSERT, UPDATE y DELETE realizadas sobre las tablas del esquema Tramite.

2. Script de la solución en T-SQL



```
USE GradosTítulos;
GO

CREATE DATABASE AUDIT SPECIFICATION AuditSpec_Tramite
FOR SERVER AUDIT Audit_AccesosGradosTítulos
ADD (INSERT, UPDATE, DELETE ON SCHEMA::Tramite BY public)
WITH (STATE = ON);
GO

-- Consulta de las modificaciones registradas sobre el esquema Tramite
SELECT event_time, server_principal_name, database_name,
       schema_name, object_name, statement
FROM sys.fn_get_audit_file('C:\SQLAudit\GradosTítulos\*.sqlaudit', DEFAULT, DEFAULT)
WHERE schema_name = 'Tramite'
ORDER BY event_time DESC;
GO
```

3. Justificación técnica de la solución aplicada

Auditar específicamente las operaciones de escritura (INSERT, UPDATE, DELETE) sobre el esquema Tramite permite trazar cambios sobre expedientes académicos, identificar quién modificó un registro y cuándo, lo cual es indispensable ante reclamos o discrepancias en el estado de un trámite de grado o título.

4. Buenas prácticas utilizadas en el proyecto

- Auditar por esquema o por tabla crítica en lugar de auditar toda la base de datos, para no generar volúmenes de log innecesarios que dificulten el análisis.
- Usar BY public para capturar la acción de cualquier usuario, evitando dejar fuera del alcance a un rol que se cree con posterioridad.
- Combinar la auditoría de escritura con la auditoría de accesos (Proyecto 1) para tener trazabilidad completa de quién entró y qué modificó.

Proyecto 3: Sistema integral de auditoría institucional

1. Enunciado del ejercicio

Diseñar una solución de auditoría que registre: inicio de sesión, cambios de permisos, creación de usuarios, eliminación de objetos y modificaciones de datos críticos.

2. Script de la solución en T-SQL



```
USE GradosTítulos;
GO

CREATE DATABASE AUDIT SPECIFICATION AuditSpec_Integral
FOR SERVER AUDIT Audit_AccesosGradosTítulos
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (FAILED_LOGIN_GROUP),
ADD (DATABASE_PERMISSION_CHANGE_GROUP),
ADD (DATABASE_PRINCIPAL_CHANGE_GROUP),
ADD (SCHEMA_OBJECT_CHANGE_GROUP),
ADD (UPDATE, DELETE ON SCHEMA::Academico BY public),
ADD (UPDATE, DELETE ON SCHEMA::Documento BY public)
WITH (STATE = ON);
GO

-- Consulta consolidada del log de auditoria institucional
SELECT event_time, action_id, succeeded, server_principal_name,
       database_name, schema_name, object_name, statement
FROM sys.fn_get_audit_file('C:\SQLAudit\GradosTítulos\*.sqlaudit', DEFAULT, DEFAULT)
ORDER BY event_time DESC;
GO
```

3. Justificación técnica de la solución aplicada

Un sistema integral de auditoría combina grupos de eventos de servidor (inicio de sesión, cambios de permisos, creación de usuarios) con auditoría puntual sobre los esquemas de mayor criticidad (Academico y Documento). Esto cubre tanto la dimensión de seguridad (quién se conecta y qué privilegios cambia) como la dimensión de integridad de los datos institucionales.

4. Buenas prácticas utilizadas en el proyecto

- Agrupar los eventos de auditoría según su naturaleza (acceso, cambios de permisos, cambios de datos) para facilitar el análisis y la generación de reportes diferenciados.
- Restringir quién puede modificar o deshabilitar la propia especificación de auditoría (SERVER AUDIT), de modo que un atacante no pueda 'apagar' la auditoría antes de actuar.
- Centralizar y respaldar periódicamente los archivos .sqlaudit fuera del servidor auditado.
- Revisar el log de auditoría de forma programada, no solo ante un incidente reportado, para detectar anomalías de forma proactiva.